

Intro and parsimony exercises FFMA

Miquel Arnedo

2025-09-28

Credits

This exercise is based on the R tutorial for course BIOS1140 at the University of Oslo and examples provided in the github of **phangorn** by Klaus Schliep

What to expect

In this tutorial, we will introduce phylogenetics as a means to visualise the evolutionary relationships among species and will conduct our first phylogenetic analyses using parsimony as method of inference. In this section we will:

- learn some tools for visualizing phylogenetic trees
- learn how to create phylogenies

Getting started

First, we need to set up our R environment. We'll load **tidyverse** a package that facilitates data manipulation and visualization, along with a few more packages today to help us handle different types of data. Chief among these is **ape** which is the basis for a lot of phylogenetic analysis in R. We will also load another phylogenetic package, **phangorn** (which has an extremely geeky reference in its name).

```
# clear the R environment
rm(list = ls())

# install new packages
install.packages("ape")
install.packages("phangorn")
install.packages("ips")
install.packages("TreeSearch")
install.packages("tidyverse")

# load packages
library(ape)
library(phangorn)
library(ips)
library(TreeSearch)
library(tidyverse)
```

With these packages installed, we are ready to begin!

—SESSION 1—

Phylogenetics in R

R has a number of extremely powerful packages for performing phylogenetic analysis, from plotting trees to testing comparative models of evolution. You can see [here](#) for more information if you are interested in learning about what sort of things are possible. For today's session, we will learn how to handle and visualize phylogenetic trees in R. We will also construct a series of trees from a sequence alignment. First, let's familiarize ourselves with how R handles phylogenetic data.

Storing trees in R

The backbone of most phylogenetic analysis in R comes from the functions that are part of the **ape** package. **ape** stores trees as **phylo** objects, which are easy to access and manipulate. The easiest way to understand this is to have a look at a simple phylogeny, so we'll create a random tree now.

```
# set seed to ensure the same tree is produced
set.seed(32)
# generate a tree
tree <- rtree(n = 4, tip.label = c("a", "b", "c", "d"))
```

What have we done here? First, the `set.seed` function just sets a seed for our random simulation of a tree. You won't need to worry about this for the majority of the time, here we are using it to make sure that when we randomly create a tree, we all create the same one.

What you need to focus on is the second line of code that uses the `rtree` function. This is simply a means to generate a random tree. With the `n = 4` argument, we are simply stating our tree will have four taxa and we are already specifying what they should be called with the `tip.label` argument.

Let's take a closer look at our `tree` object. It is a **phylo** object - you can demonstrate this to yourself with `class(tree)`.

```
tree
#>
#> Phylogenetic tree with 4 tips and 3 internal nodes.
#>
#> Tip labels:
#>   c, a, d, b
#>
#> Rooted; includes branch length(s).
```

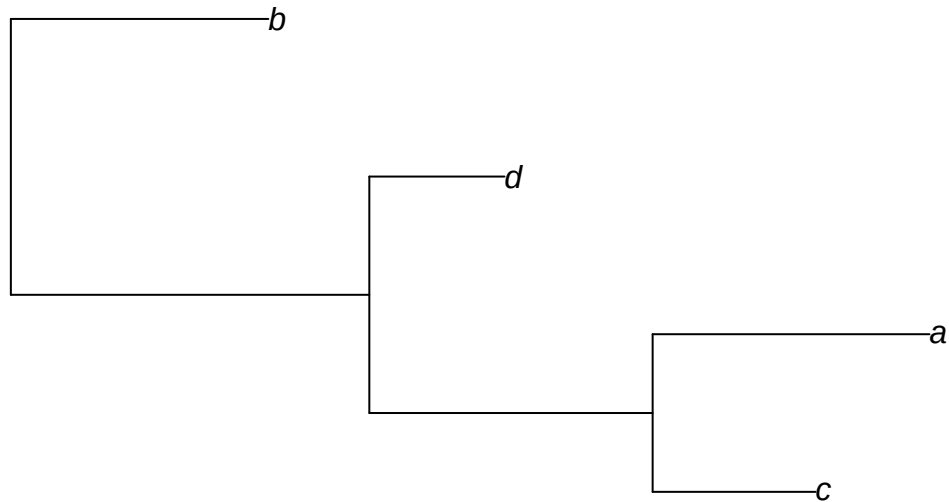
By printing `tree` to the console, we see it is a tree with 4 tips and 3 internal nodes, a set of tip labels. We also see it is rooted and that the branch lengths are stored in this object too.

You can actually look more deeply into the data stored within the `tree` object if you want to. Try the following code and see what is inside.

```
str(tree)
objects(tree)
tree$edge
tree$edge.length
```

It is of course, much easier to understand a tree when we visualise it. Luckily this is easy in R.

```
plot(tree)
```



In the next section, we will learn more about how to plot trees.

Plotting trees

We can do a lot with our trees in R using a few simple plot commands. We will use some of these later in the tutorial and assignment, so here's a quick introduction of some of the options you have.

First, let's generate another random tree, this time with 5 taxa.

```
# set seed to ensure the same tree is produced
set.seed(32)
# generate a tree
tree <- rtree(n = 5, tip.label = c("a", "b", "c", "d", "e"))
```

Now, try modifying the appearance of the tree using some of these arguments to `plot()`:

- `use.edge.length` (TRUE (default) or FALSE): should branch length be used to represent evolutionary distance?
- `type`: the type of tree to plot. Options include “phylogram” (default), “cladogram”, “unrooted” and “fan”.
- `edge.width`: sets the thickness of the branches
- `edge.color`: sets the color of the branches

See `?plot.phylo` for a comprehensive list of arguments.

You can also manipulate the contents of your tree:

- `drop.tip()` removes a tip from the tree
- `rotate()` switches places of two tips in the visualisation of the tree (without altering the evolutionary relationship among taxa)
- `extract.clade()` subsets the tree to a given clade

See the help pages for the functions to find out more about how they work. Now, let's use some of the options we've learned here for looking at some real data.

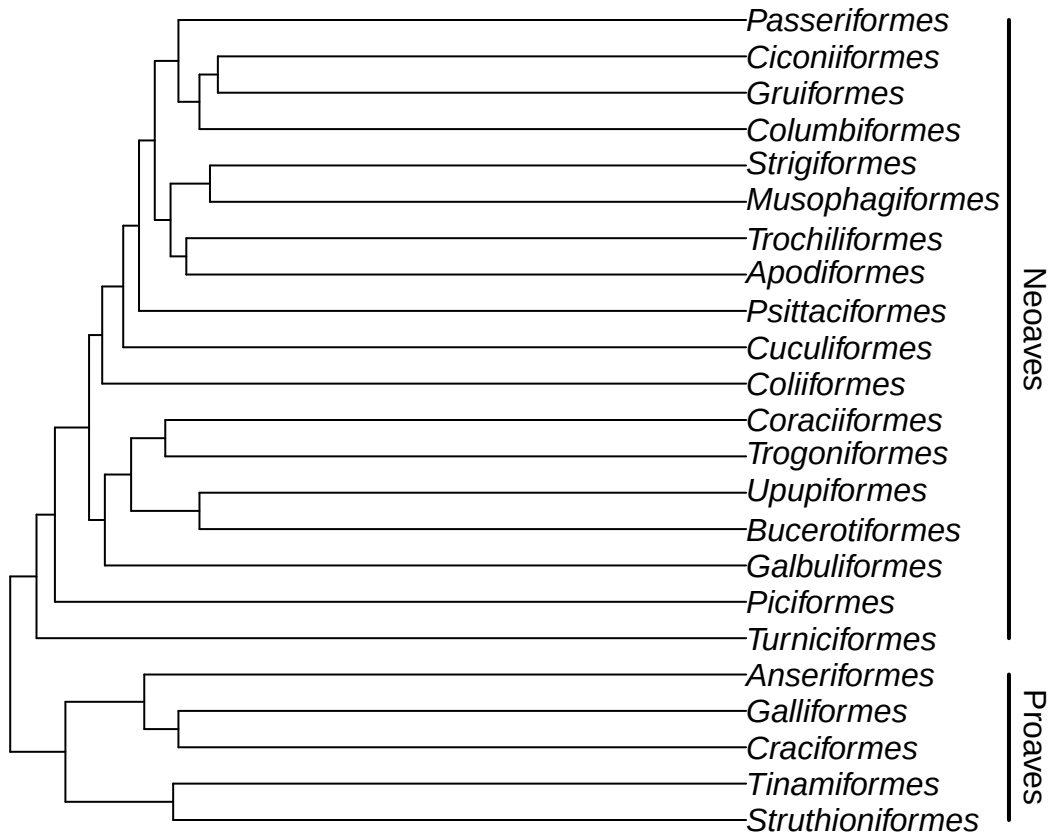
A simple example with real data - avian phylogenetics

So far, we have only looked at randomly generated trees. Let's have a look at some data stored within `ape`—a phylogeny of birds at the order level.

```
# get bird order data  
data("bird.orders")
```

Let's plot the phylogeny to have a look at it. We will also add some annotation to make sense of the phylogeny.

```
# no.margin = TRUE gives prettier plots  
plot(bird.orders, no.margin = TRUE)  
segments(38, 1, 38, 5, lwd = 2)  
text(39, 3, "Proaves", srt = 270)  
segments(38, 6, 38, 23, lwd = 2)  
text(39, 14.5, "Neoaves", srt = 270)
```



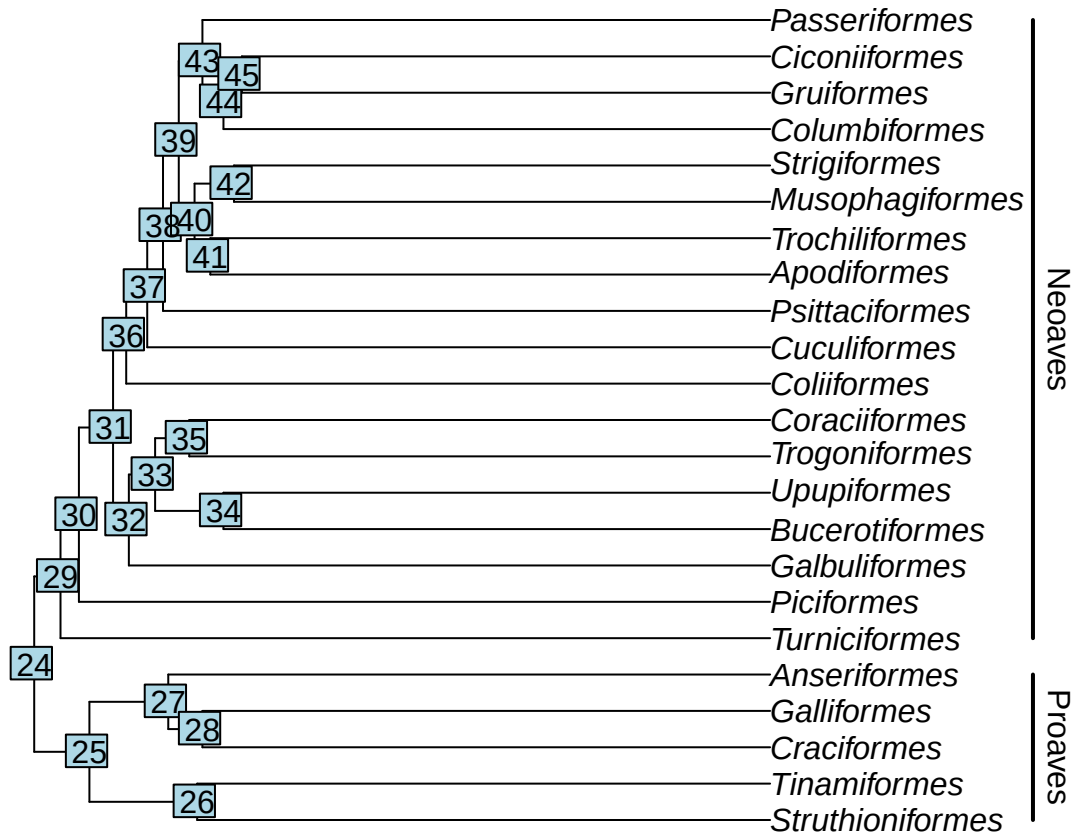
Here, the `segments` and `text` functions specify the bars and names of the two major groups in our avian phylogeny. We are just using them for display purposes here, but if you'd like to know more about them, you can look at the R help with `?segments` and `?text` commands.

Let's focus on the Neoaves clade for now. Perhaps we want to test whether certain families within Neoaves form a monophyletic group? We can do this with the `is.monophyletic` function.

```
# Parrots and Passerines?
is.monophyletic(bird.orders, c("Passeriformes", "Psittaciformes"))
#> [1] FALSE
# hummingbirds and swifts?
is.monophyletic(bird.orders, c("Trochiliformes", "Apodiformes"))
#> [1] TRUE
```

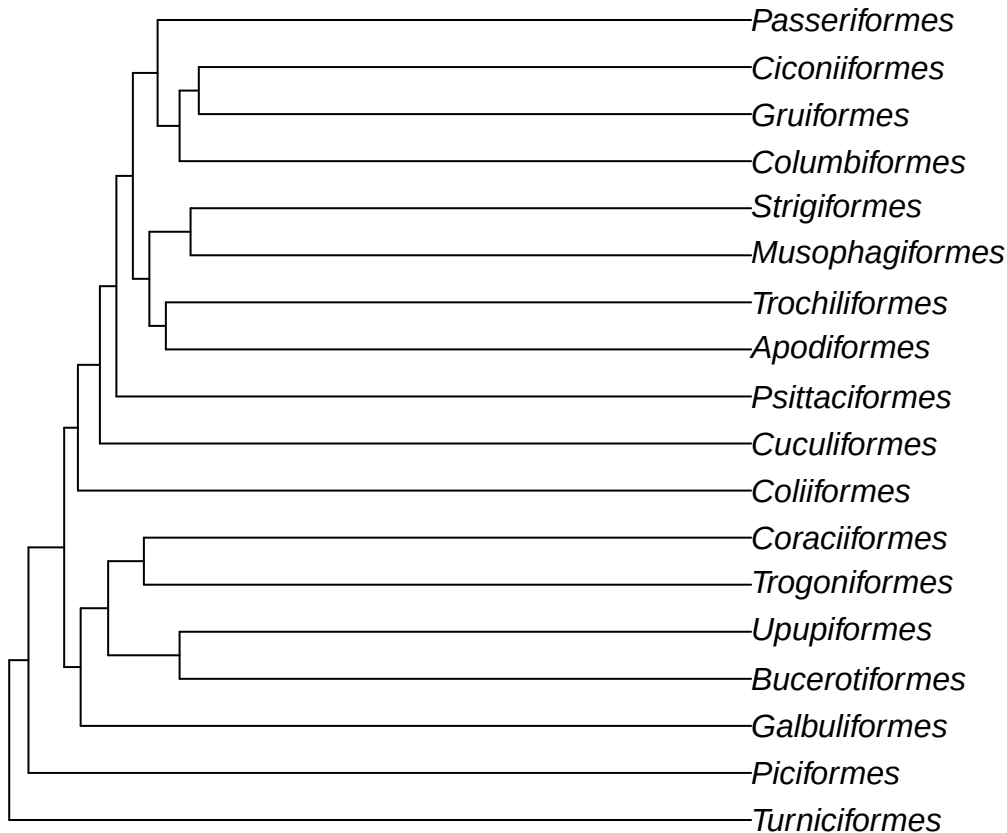
If we want to look at just the Neoaves, we can subset our tree using `extract.clade()`. We need to supply a node from our tree to `extract.clade`, so let's find the correct node first. The nodes in the tree can be found by running the `nodelabels()` function after using `plot()`:

```
plot(bird.orders, no.margin = TRUE)
segments(38, 1, 38, 5, lwd = 2)
text(39, 3, "Proaves", srt = 270)
segments(38, 6, 38, 23, lwd = 2)
text(39, 14.5, "Neoaves", srt = 270)
nodelabels()
```



We can see that the Neoaves start at node 29, so let's extract that one.

```
# extract clade
neoaves <- extract.clade(bird.orders, 29)
# plot
plot(neoaves, no.margin = TRUE)
```



The functions provided by **ape** make it quite easy to handle phylogenies in R, feel free to experiment further to find out what you can do!

Inferring trees with R using parsimony

So far, we have only looked at examples of trees that are already constructed in some way. However, if you are working with your own data, this is not the case - you need to actually make the tree yourself. Luckily, **phangorn** is ideally suited for this. We will use some data, bundled with the package, for the next steps. In this example, we will investigate the phylogenetic relationships of *Parachtes* a genus belonging to the spider family Dysderidae by using a concatenated matrix of 6 mtDNA genes, namely *cox1*, *nad1*, *16S* and *12S* and 3 nuclear genes, *18S*, *28S* and *Histone3*, obtained from Genbank.

The following code loads the data as phyDat object:

```
# get parachtes data
parachtes <- read.phyDat("ParALL153.fas", format = "fasta")
```

Tree search

There are three different search strategies:

1. **Exhaustive search** (sometimes also referred as implicit enumeration, e.g. in TNT): It does guarantee the shortest tree, but there is usually a taxa limit, depending on the program used (~15). In the case of **phangorn** the usage would be `bab(data, tree = NULL, trace = 0, ...)`

2. Heuristic search: does not guarantee finding the shortest tree. It usually consists of two rounds: (1) you build a tree (using different strategies, e.g., random addition of taxa, Wagner tree, etc.), (2) you conduct *branch swapping*, an algorithm that exchange branches of the initial tree to look for shorter trees. There are different branch swapping strategies, ranging from lighter to more thorough rearrangements: NNI, SPR, TBR. In **phangorn**, you can use **random.addition** to compute a starting tree. The function **optim.parsimony** performs tree rearrangements to find trees with a lower parsimony score. The tree rearrangements implemented are nearest-neighbor interchanges (NNI) and subtree pruning and regrafting (SPR). The latter so far only works with the Fitch algorithm. We iterate these procedures many times (e.g., more than 100).

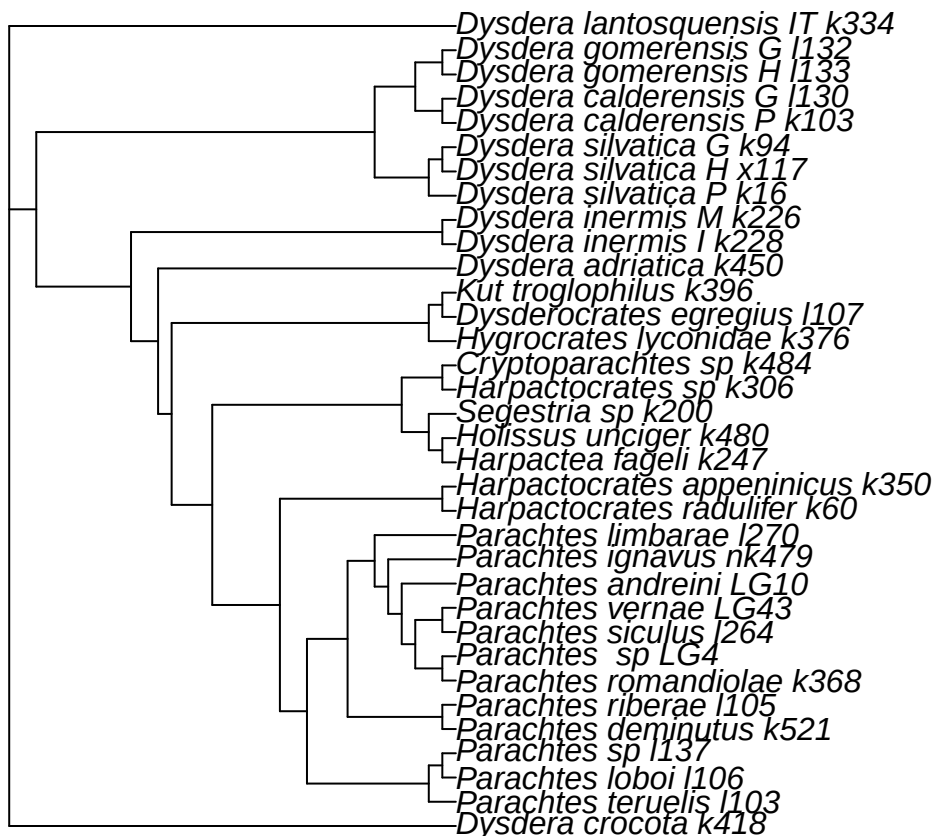
3. **New search strategies:** these are optimised algorithm for heuristic searchers of large data matrices (e.g. >100 taxa). There are several strategies. In this practical, we will implement *parsimony ratchet*. The function is **pratchet**, an implementation of the parsimony ratchet (Nixon 1999). This allows to escape local optima and find better trees than only performing NNI / SPR rearrangements.

The current implementation is:

1. Create a bootstrap data set from the original data set.
2. Take the current best tree and perform tree rearrangements on and save bootstrap tree as .
3. Use and perform tree rearrangements on the original data set. If this tree has a lower parsimony score than the currently best tree, replace it.
4. Iterate 1:3 until either a given number of iteration is reached (minit) or no improvements have been recorded for a number of iterations (k).

Now that we have inferred the MP tree, we should plot it to have a look.

```
# plot MP tree
plot(treeRatchet, no.margin = TRUE)
```



Tree rooting

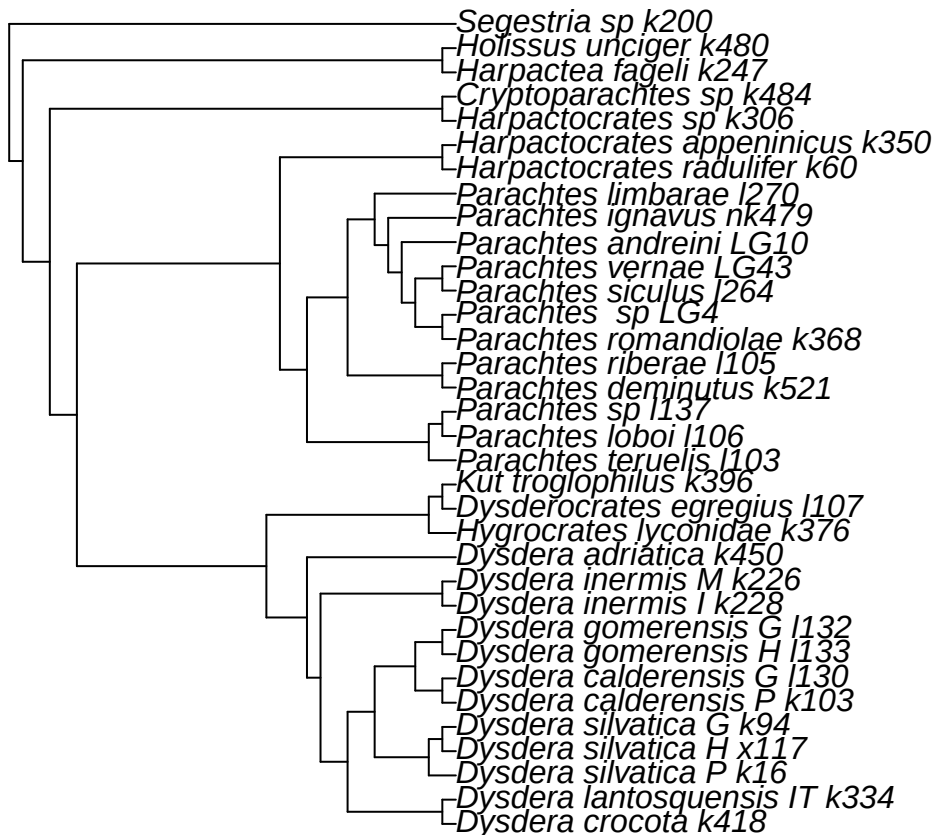
Notice that so far we have not define which species is the outgroup for the analysis. By default the first taxon in the matrix is assigned as outgroup. Also, notice that even if show the tree as rooted, the analyses infer trees that are unrooted. We can verify that the tree is unrooted using the `is.rooted()` function.

```
# check whether the tree is rooted
is.rooted(treeRatchet)
```

We can also set a root on our tree, if we know what we should set the outgroup to. In our case, we can set our outgroup to **Segestria_sp_k200**.

We will set the root of our tree using the `root` function and we'll then plot it to see how it looks.

```
# plot treeRatchet rooted
treeRatchet_r <- root(treeRatchet, "Segestria_sp_k200", resolve.root = TRUE, edgelabel = TRUE)
plot(treeRatchet_r, no.margin = TRUE)
```



Notice that the tree remains the same, you can move the root to any other node, and the tree is fully equivalent. You can check that by asking again about the length of the tree with the new root.

```
# reporting tree length (steps)
parsimony(treeRatchet_r, parachtes)
#> [1] 7893
```

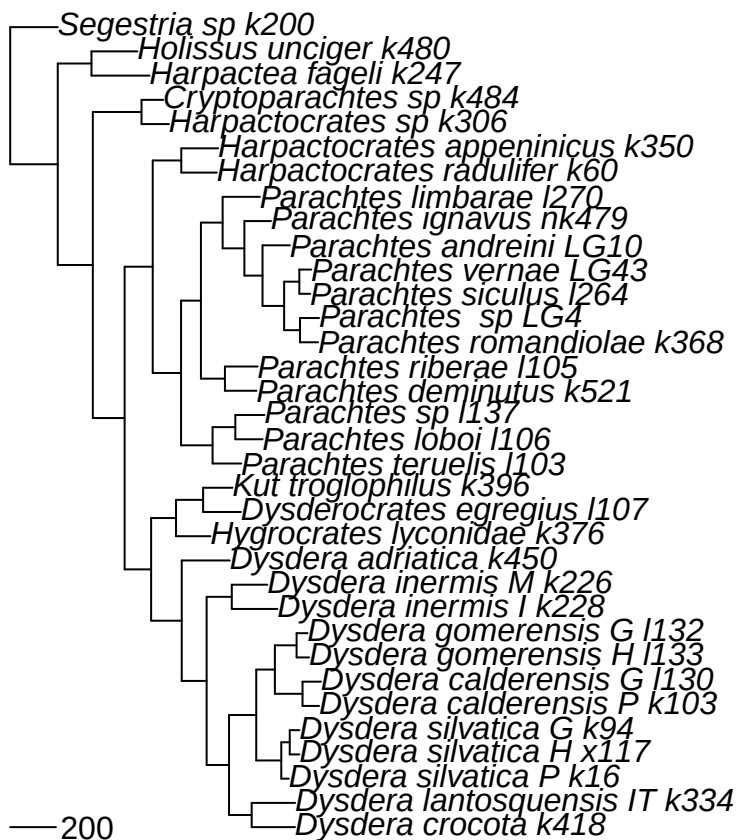
Branch lengths

Also, observe that this is tree only inform about the topology. If we wanted to also include the number of substitution in each branch (i.e. branch length), we have to do the following:

```
# perform character optimization
treeRatchet_r<-acctran(treeRatchet_r, parachtes)
```

This function assigns edge weights. We now plot the tree with the corresponding branch length information:

```
# plot the rooted tree with branch lengths
plot(treeRatchet_r, type="phylogram", no.margin = TRUE)
add.scale.bar()
```



Exporting a tree

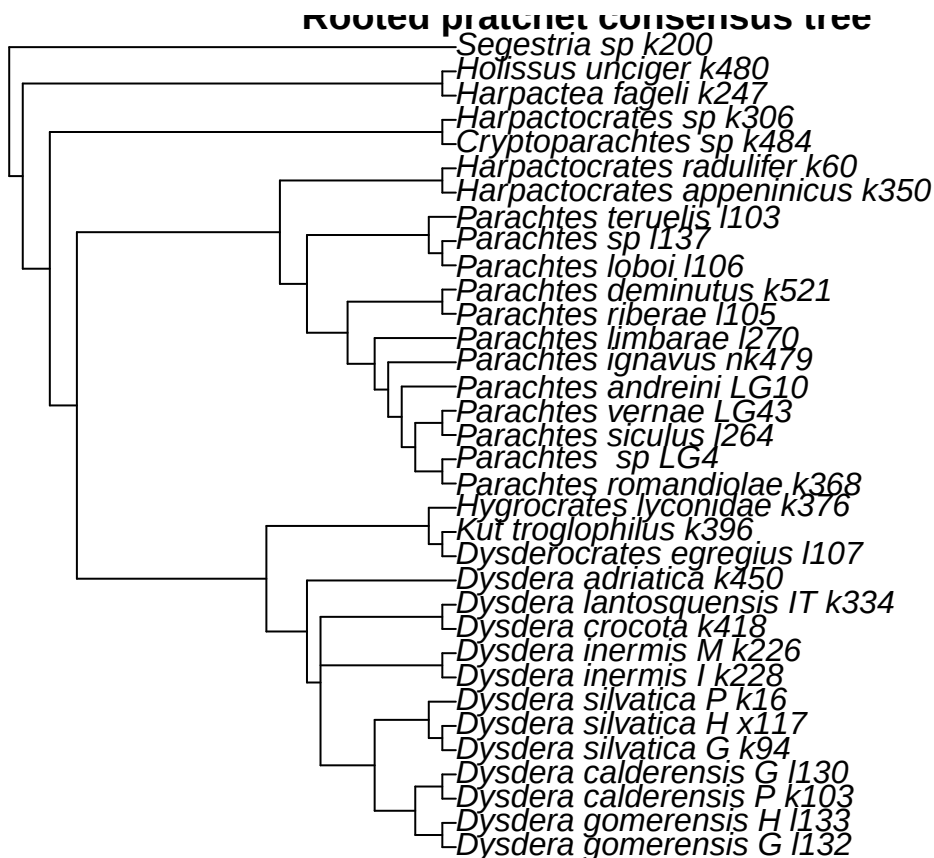
Now that we have our tree properly rooted and with branch lengths, we can easily write it to a file in Newick format:

```
# rooted tree with branch lengths
write.tree(treeRatchet_r, "parachtes.tre")
```

Sometimes we find more than one equally parsimonious tree. In those cases, we usually calculate the strict consensus tree, which helps visualize the conflict among the best trees. In our case, we found one single shortest tree, but we can pretend to have a second one by generating a new tree using random addition of taxa, which is generally suboptimal, and a second one optimizing the first with an SPR swapper.

```
# compare the length of the trees
parsimony(c(treeRA, treeSPR), parachtes)
#> [1] 7894 7893
```

```
#combines both trees into a single object
obj<-c(treeRA, treeSPR)
# Calculate and plot the consensus tree
obj_cons <- root(consensus(obj), outgroup = "Segestria_sp_k200", resolve.root = TRUE, edgelabel = TRUE)
plot(obj_cons, main="Rooted pratchet consensus tree", no.margin = TRUE)
```



11

—SESSION 2—

Gaps

So far we have conducted all parsimony analyses assuming gaps are missing data, which is the default option. However, we may want to investigate if our inferences would change if gaps were scored as an additional character state. We can do this by using the following commands:

```
#indicate gaps defined as "-" are a new state and that "n" and "?" should be considered missing data
parachtes_5<-gap_as_state(parachtes, gap = "-", ambiguous = c("n","?"))
```

Similarly, we can decide to score gaps as an alternative absence/presence character, following the simple coding method proposed by Simmons & Ochoterena. 2000. We will use the `ipspackage` command `code.simple.gaps`. Note that the gapped positions are excluded from the matrix, and therefore the new matrix would have a different number of characters

```
#First, we will transform our parachtes matrix from a phyDat class object to a DNABin class object
parachtes_DNABin<-as.DNABin(parachtes)
#Now we can recode the gaps as absence/presence data
parachtes_DNABin_ap<-code.simple.gaps(parachtes_DNABin)
#> $Gap_1
#> [1] 1952
#>
#> $Gap_2
#> [1] 1984
#>
#> $Gap_3
#> [1] 2532
#>
#> $Gap_4
#> [1] 2670
#>
#> $Gap_5
#> [1] 2800 2801
#>
#> $Gap_6
#> [1] 2929
#>
#> $Gap_7
#> [1] 2935
#>
#> $Gap_8
#> [1] 3029
#>
#> $Gap_9
#> [1] 3642 3643
#>
#> $Gap_10
#> [1] 3675
#>
#> $Gap_11
#> [1] 3691
#>
```

```
#> $Gap_12
#> [1] 3709 3710
#>
#> $Gap_13
#> [1] 3715
#>
#> $Gap_14
#> [1] 3728
#>
#> $Gap_15
#> [1] 3737 3738
#>
#> $Gap_16
#> [1] 3767 3768 3769
#>
#> $Gap_17
#> [1] 3798
#>
#> $Gap_18
#> [1] 3840
#>
#> $Gap_19
#> [1] 3862
#>
#> $Gap_20
#> [1] 3923
#>
#> $Gap_21
#> [1] 3965
#>
#> $Gap_22
#> [1] 3987
#>
#> $Gap_23
#> [1] 4024 4025
#>
#> $Gap_24
#> [1] 4027
#>
#> $Gap_25
#> [1] 4182
#>
#> $Gap_26
#> [1] 4187
#>
#> $Gap_27
#> [1] 4191
#>
#> $Gap_28
#> [1] 4194 4195
#>
#> $Gap_29
#> [1] 4234
```

```
#>
#> $Gap_30
#> [1] 4260
#>
#> $Gap_31
#> [1] 4288 4289
#>
#> $Gap_32
#> [1] 4298
#>
#> $Gap_33
#> [1] 4304
#Finally, we will transform the new matrix with the recoded gaps back into a phyDat object
parachtes_ap<-as.phyDat(parachtes_DNAbin_ap)
```

The new coding has generated 33 additional characters, and has removed 42 positions of the original 4398

Node support

Inference methods like Parsimony and Maximum Likelihood produce one or more best trees that meet certain optimality criteria, such as being the shortest tree or best explaining the data given a specific evolutionary model. However, we may need to assess the impact of character undersampling (random error) on our results. Specifically, we want to determine the support for the nodes recovered in the preferred tree(s). One way to do this is by using **resampling techniques**.

These techniques involve generating pseudoreplicates (ideally >100) of our original matrix either by resampling characters with repetition (*bootstrap*) or by randomly removing a proportion of the original matrix (*Jaccknife*). For each matrix, we find the best tree(s). Finally, we determine the node support by estimating the proportion of time that each node is recovered in the resampled trees. We will estimate node support using resampling with the package **TreeSearch**.

The package includes a more intensive version of the parsimony ratchet strategy for estimating the most parsimonious tree. we will first implement this strategy using the following commands:

Node support based on Jackknife resampling

We will now estimate the node support using Jackknife. This is similar to Bootstrap, the most widely resampling method for finding node support. It just differs by the type of resampling implemented, removal instead of resampling with repetition. If you want to use bootstrap instead, just replace `method = "jack"` by `method = "bootstrap"`.

```
# Jackknife resampling, we will build 10 resampled matrices (pseudoreplicates) only due to time constraints
nReplicates <- 10
jackTrees <- replicate(nReplicates,
  #c() ensures that each replicate returns a list of trees
  c(Resample(parachtes, method = "jack", proportion = 2 / 3, ratchIter = 2, tbrIter = 2, startIter = 1,
    maxHits = 5, maxTime = 1 / 10, verbosity = 0))
)
```

Now we must decide what to do with the multiple optimal trees from each replicate. Alternatives are:

1. Treat each tree equally

```
JackLabels(ape::consensus(trees), unlist(jackTrees, recursive = FALSE))
```

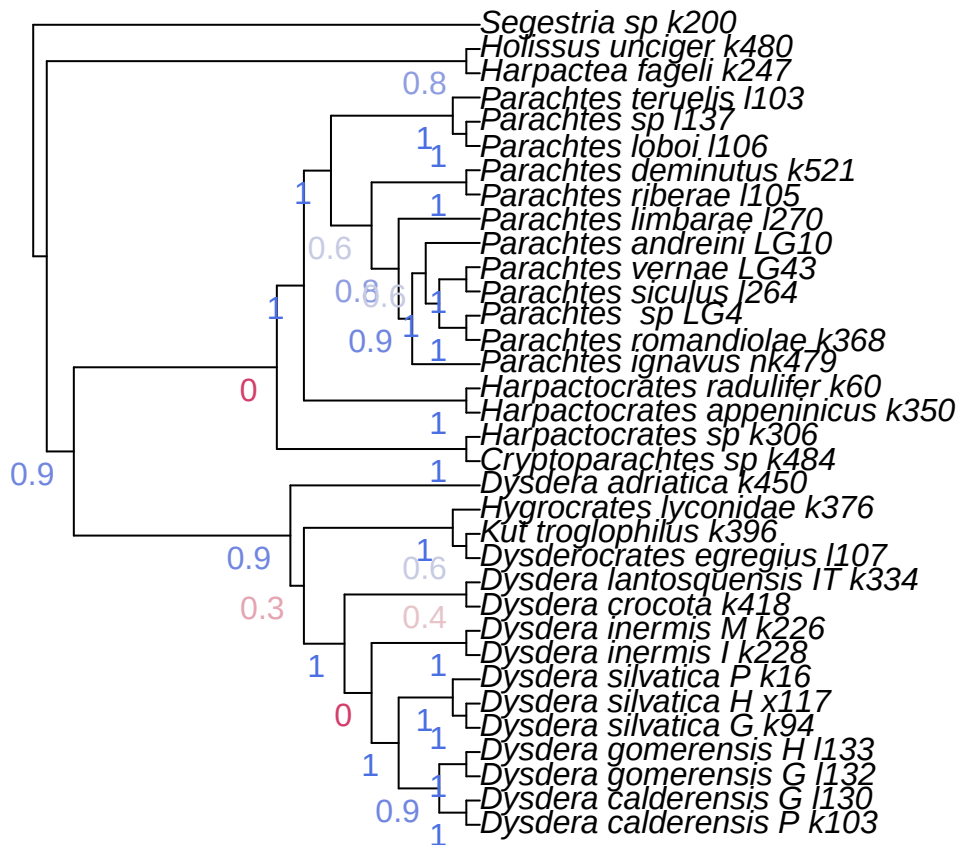
```
JackLabels(ape::consensus(trees), lapply(jackTrees, ape::consensus))
```

```
JackLabels(ape::consensus(trees), lapply(jackTrees,[[, 1))
```

```
jackTrees_consensus<-lapply(jackTrees, consensus)
```

```
jackTrees_consensus_r <- lapply(jackTrees_consensus, function(tree) {
  root(tree, "Segestria_sp_k200", resolve.root = TRUE, edgelabel = TRUE)
})
```

```
plot(parachtes_tre_r[[1]], no.margin = TRUE)
```



Test congruence among data partitions via ILD test

There is currently no R function to perform the ILD test in any R package. We will create a function (`ild_test_dnabin`) using the following R script:

```
# Main ILD (Farris PHT) style test:
# - aln_bin: DNABin alignment (rows = taxa, cols = sites; same taxa order across all sites)
# - parts: named list of integer vectors of site indices (original column positions)
# - nperm: number of permutations (e.g., 99, 199, 999)
# - pratchet_iter: iterations for MP search (balance speed/quality)
# Returns observed stat, null distribution, and p-value.

ild_test_dnabin <- function(aln_bin, parts, nperm = 99, seed = 1,
                           pratchet_iter = 50, tree_comb = NULL, quiet = FALSE) {
  set.seed(seed)

  # ----- checks -----
  if (!inherits(aln_bin, "DNABin"))
    stop("aln_bin must be DNABin (use ape::read.dna).")

  L <- ncol(aln_bin)
  if (is.null(L) || L < 2) stop("Alignment must have >= 2 sites.")
  if (!is.list(parts) || length(parts) < 2)
    stop("Provide >= 2 partitions in 'parts'.")

  sizes <- vapply(parts, length, 1L)
  if (sum(sizes) != L) {
    stop(sprintf("Sum of partition sizes (%d) != alignment length (%d).", sum(sizes), L))
  }

  # Bounds check
  max_idx <- max(unlist(parts))
  min_idx <- min(unlist(parts))
  if (min_idx < 1 || max_idx > L) {
    stop(sprintf("Partition indices out of bounds: allowed 1..%d, got [%d..%d].", L, min_idx, max_idx))
  }

  # Helper for MP length
  mp_tree_length <- function(aln_bin, pratchet_iter = 50, fixed_tree = NULL) {
    dat <- as.phyDat(aln_bin)
    if (is.null(fixed_tree)) {
      tr <- pratchet(dat, maxit = pratchet_iter)
    } else {
      tr <- fixed_tree
    }
    parsimony(tr, dat)
  }

  # ----- observed statistic -----
  # Combined tree length: use precomputed if given
  if (!is.null(tree_comb)) {
    if (!inherits(tree_comb, "phylo"))
      stop("tree_comb must be a 'phylo' object.")
  }
}
```



```

    TL_comb <- mp_tree_length(aln_bin, fixed_tree = tree_comb)
  } else {
    TL_comb <- mp_tree_length(aln_bin, pratchet_iter)
  }

  # Sum of MP lengths from separate partitions (each optimized separately)
  TL_sep <- sum(sapply(parts, function(p)
    mp_tree_length(aln_bin[, p, drop = FALSE], pratchet_iter)))

  stat_obs <- TL_sep - TL_comb
  if (!quiet) cat(sprintf("Observed (TL_sep - TL_comb) = %.3f\n", stat_obs))

  # ----- permutation test -----
  all_sites <- unlist(parts, use.names = FALSE)
  null_vals <- numeric(nperm)
  pb <- if (!quiet) txtProgressBar(min = 0, max = nperm, style = 3) else NULL

  for (i in seq_len(nperm)) {
    perm <- sample(all_sites, length(all_sites), replace = FALSE)
    # repartition
    idx_list <- vector("list", length(sizes))
    start <- 1
    for (k in seq_along(sizes)) {
      idx_list[[k]] <- perm[start:(start + sizes[k] - 1)]
      start <- start + sizes[k]
    }
    # recompute TL_sep_perm
    TL_sep_perm <- sum(sapply(idx_list, function(p)
      mp_tree_length(aln_bin[, p, drop = FALSE], pratchet_iter)))
    null_vals[i] <- TL_sep_perm - TL_comb
    if (!quiet) setTxtProgressBar(pb, i)
  }
  if (!quiet) close(pb)

  # Right-tailed p-value as in PAUP*: proportion of permuted >= observed

  p_val <- (sum(null_vals >= stat_obs) + 1) / (nperm + 1)

  out <- list(stat_obs = stat_obs, null = null_vals, p_value = p_val,
    TL_sep = TL_sep, TL_comb = TL_comb,
    nperm = nperm, sizes = sizes,
    tree_comb = tree_comb)
  class(out) <- "ILDdnabin"
  out
}

# Pretty print
print.ILDdnabin <- function(x, ...) {
  cat("ILD (DNABin) permutation test\n")
  cat(sprintf("  Observed: TL_sep - TL_comb = %.3f\n", x$stat_obs))
  cat(sprintf("  Permutations: %d\n", x$nperm))
  cat(sprintf("  p-value (right-tailed): %.4f\n", x$p_value))
  invisible(x)
}

```

```

}

# Quick plot for the null distribution
plot.ILDdnabin <- function(x, ...) {
  hist(x$null, breaks = "FD", main = "ILD null distribution",
       xlab = "TL_sep_perm - TL_comb", col = "grey80", border = "white")
  abline(v = x$stat_obs, col = "red", lwd = 2)
  mtext(sprintf("Observed = %.3f; p = %.4f", x$stat_obs, x$p_value), col = "red", line = 0.5)
}

```

1. We will read the alignment into a DNAbin object because phyDat format condenses the columns by patterns, which makes it difficult to assign original partitions

```
aln_bin <- read.dna("ParALL153.fas", format = "fasta")
```

2. If we already inferred the most parsimonious tree for the overall alignment, we can just use the information.

```
mp_tree_comb <- parsimony(parachtes_tre, parachtes)
```

or, alternatively we could recalculate the tree length for the combined data matrix via pratchet()

```

#dat_full <- as.phyDat(aln_bin)
#mp_tree_comb <- pratchet(dat_full, maxit = 100)

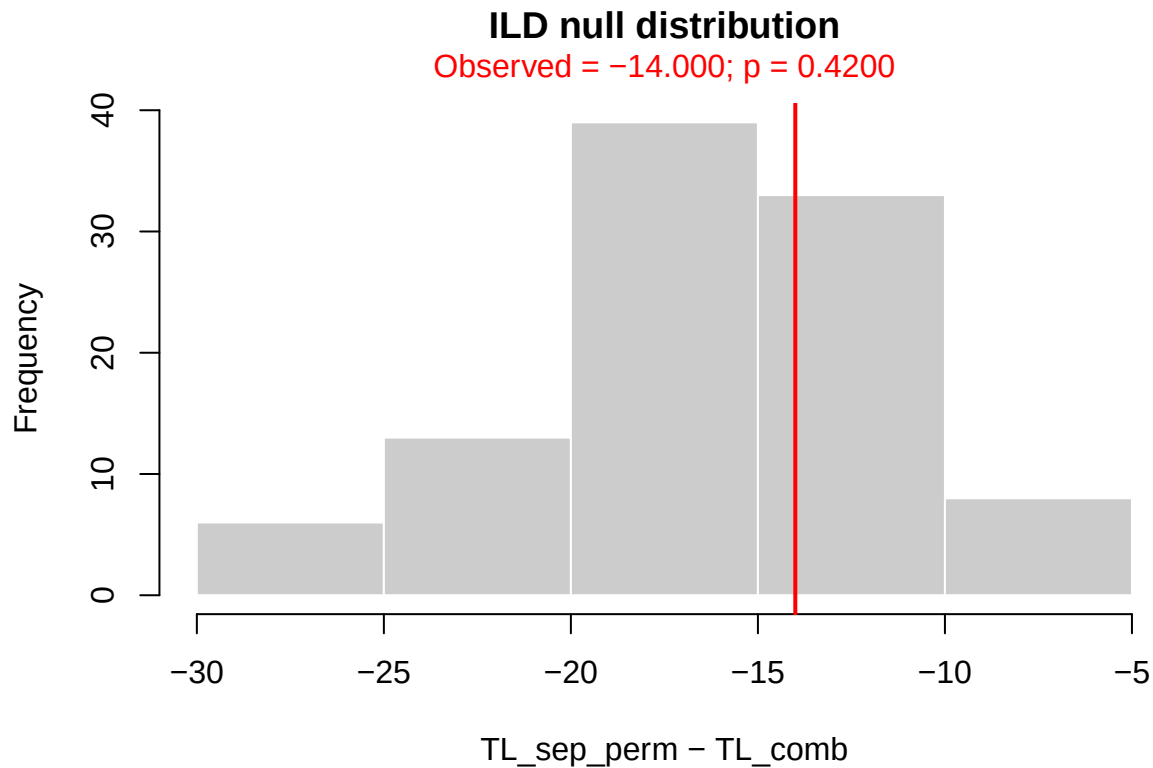
```

3. We will define partitions by ORIGINAL column indices (must cover all columns). In this example, we will compare the mtDNA genes (COI, 12S, 16S, which correspond to the first 2731 positions) versus the nuc genes (18S, 28S, H3)
4. We can now run the ILD test. Note that we have to define the number of permutations (nperm); the more, the better—usually 1000 is the minimum. However, in this example, to avoid spending too much time, we will only run 100 permutations (99 plus the observed partition lengths).
5. We can now inspect the results

```

print(res)
#> ILD (DNAbin) permutation test
#>   Observed: TL_sep - TL_comb = -14.000
#>   Permutations: 99
#>   p-value (right-tailed): 0.4200
plot(res)

```



Exercises

Find the most parsimonious tree, considering gaps as absence/presence. Properly root the tree and report the tree, indicating the number of steps (length). Assess the node support using Jackknife. Finally, test if the moleculars and the recoded gap partitions are congruent.

In the CV, use the task option to submit your answer

Session info

```
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 15.6.1
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK v
#>
#> locale:
#> [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
#>
#> time zone: Europe/Madrid
#> tzcode source: internal
#>
```

```

#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] TreeSearch_1.6.0 ips_0.0.12      phangorn_2.12.1 ape_5.8-1      lubridate_1.9.4
#> [6] forcats_1.0.0   stringr_1.5.1   dplyr_1.1.4     purrr_1.0.4    readr_2.1.5
#> [11] tidyr_1.3.1     tibble_3.2.1    ggplot2_3.5.2   tidyverse_2.0.0
#>
#> loaded via a namespace (and not attached):
#> [1] tidyselect_1.2.1   farver_2.1.2      R.utils_2.13.0    bitops_1.0-9
#> [5] fastmap_1.2.0      RCurl_1.98-1.17   promises_1.3.2     PlotTools_0.3.1
#> [9] shinyjs_2.1.0      XML_3.99-0.18     digest_0.6.37      timechange_0.3.0
#> [13] mime_0.13          lifecycle_1.0.4   cluster_2.1.8.1    magrittr_2.0.3
#> [17] compiler_4.4.1     rlang_1.1.6       tools_4.4.1        TreeTools_1.14.0
#> [21] igraph_2.1.4       yaml_2.3.10       data.table_1.17.2  knitr_1.50
#> [25] bit_4.6.0          plyr_1.8.9        RColorBrewer_1.1-3 R.cache_0.17.0
#> [29] withr_3.0.2        R.oo_1.27.1       grid_4.4.1         xtable_1.8-4
#> [33] colorspace_2.1-1   future_1.49.0     globals_0.18.0     scales_1.4.0
#> [37] cli_3.6.5          rmarkdown_2.29    generics_0.1.4     RcppParallel_5.1.10
#> [41] rstudioapi_0.17.1  tzdb_0.5.0        cachem_1.1.0       Rogue_2.1.6
#> [45] parallel_4.4.1     matrixStats_1.5.0 vctrs_0.6.5        Matrix_1.7-3
#> [49] TreeDist_2.9.2     hms_1.1.3         bit64_4.6.0-1      listenv_0.9.1
#> [53] protoclust_1.6.4   parallelly_1.44.0 glue_1.8.0          codetools_0.2-20
#> [57] stringi_1.8.7      gtable_0.3.6      later_1.4.2        quadprog_1.5-8
#> [61] pillar_1.10.2      htmltools_0.5.8.1 R6_2.6.1           zigg_0.0.2
#> [65] Rdpack_2.6.4       evaluate_1.0.3    shiny_1.10.0       lattice_0.22-7
#> [69] rbibutils_2.3      R.methodsS3_1.8.2 Rfast_2.1.5.1      memoise_2.0.1
#> [73] httpuv_1.6.16      Rcpp_1.0.14       fastmatch_1.1-6    nlme_3.1-168
#> [77] xfun_0.52          fs_1.6.6          pkgconfig_2.0.3

```